Методи дерев рішень, класифікації та прогнозування

Метод дерев рішень (decision trees) є одним із найбільш популярних методів розв’язання задач класифікації й прогнозування. Іноді цей метод Data Mining також називають деревами вирішальних правил, деревами класифікації і регресії.

Якщо залежна, тобто цільова змінна приймає дискретні значення, за допомогою методу дерева рішень розв’язується задача класифікації.

Якщо ж залежна змінна приймає безперервні значення, то дерево рішень установлює залежність цієї змінної від незалежних змінних, тобто розв’язує задачу чисельного прогнозування.

У найбільш простому вигляді дерево рішень – це спосіб показу правил в ієрархічній, послідовній структурі. Основа такої структури – відповіді «Так» або «Ні» на низку питань.

Приклад

Наведемо приклад дерева рішень, задача якого – відповістити на запитання: «Чи грати в гольф?»

Щоб розв’язати задачу, тобто прийняти рішення, чи грати в гольф, слід віднести поточну ситуацію до одного з відомих класів (у цьому випадку – «грати» або «не грати»). Для цього потрібно відповісти на низку питань, які є у вузлах цього дерева, починаючи з його кореня.

Перший вузол такого дерева може бути,напиклад, «Сонячно?» і є вузлом перевірки, тобто умовою.

При позитивній відповіді на запитання здійснюється перехід до лівої частини дерева, що називається лівою гілкою, при негативному – до правої частини дерева. Отже, внутрішній вузол дерева є вузлом перевірки певної умови. Далі йде наступне питання і т.д., поки не буде досягнутий кінцевий вузол дерева, що є вузлом розв’язку. Для цього дерева існує два типи кінцевого вузла: «грати» і «не грати» у гольф.

У результаті проходження від кореня дерева (іноді називається кореневою вершиною) до його вершини розв’язується задача класифікації, тобто вибирається один із класів – «грати» чи «не грати» у гольф.

Метою побудови дерева рішень в нашому випадку є визначення значення категоріальної залежної змінної.

Для цієї задачі основними елементами дерева рішень є:
- Корінь дерева: «Сонячно?»
- Внутрішній вузол дерева або вузол перевірки: «Спекртно?», «Чи йде дощ?»
- Листок, кінцевий вузол дерева, вузол розв’язку або вершина: «Грати», «Не грати». Гілки дерева (варіанти відповіді): «Так», «Ні».

У розглянутому прикладі розв’язується задача бінарної класифікації, тобто створюється дихотомічна класифікаційна модель. Приклад демонструє роботу так званих бінарних дерев.

У вузлах бінарних дерев розгалуження може відбуватися тільки у двох напрямках, тобто існує можливість тільки двох відповідей на поставлене питання («так» і «ні»)

Дерево рішень – модель, у певному сенсі близька до моделі прийняття рішень людиною.

Наприклад, під час переходу проїжджої частини ми подумки "відповідаємо на питання" типу: який сигнал світлофора горить? чи є поблизу транспортні засоби, що рухаються? яка відстань до них? і так далі. Ці питання можуть утворювати ієрархію.

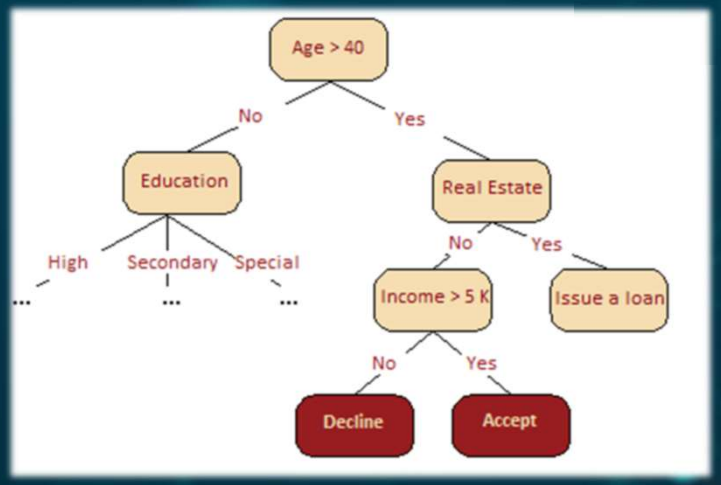
Наприклад, якщо горить червоне світло, ми приймаємо рішення "не переходити проїжджу частину", а у разі зеленого сигналу "задаємо додаткові питання" про те, чи безпечно переходити вулицю.

Схожу ідею можна використовувати (і вона справді використовувалася раніше!) для класифікації позичальників банком. Ця задача називається задачею кредитного скорингу.

Співробітник банку ставить питання позичальнику і за його відповідями на певному етапі вирішує, чи доцільно видати кредит такому позичальнику.

Той самий процес відбувається, наприклад, при постановці діагнозу у лікаря.

Подібний процес ухвалення рішення прийнято візуалізувати у вигляді деревоподібної структури (блок-схеми):



Модель дерева рішень у машинному навчанні фактично реалізує ту саму ідею. Поетапно будується серія логічних правил виду "якщо значення ознаки 1 більше 5000, про віднести об'єкт до класу 1", "якщо значення ознаки 2 дорівнює нулю, то перевірити значення ознаки 3... ". На цьому шляху природно виникають такі питання:

- як вибрати ознаку для розгалуження;
- як вибрати граничне значення для цієї ознаки.

Процес побудови дерева рішень

Для побудови гарного (і не дуже розлогого) дерева рішень важливо задати "правильні" питання про ознаки датасета. Практичною аналогією тут є гра "Вгадай об’єкт". Ведучий загадує деякий об’єкт живої природи або річ, а учасники, ставлячи запитання з відповідями "так/ні", намагаються відгадати.

У цій ситуації поставити питання "Це кішка?" менш корисно, ніж запитати "Це жива істота?" У разі негативної відповіді на перше запитання ви скоротите множину можливих варіантів лише на одиницю. При цьому будь-яка відповідь на друге питання скорочує простір пошуку приблизно вдвічі. Прийняття рішень комп'ютером у таких ситуаціях ґрунтується на ентропії.

Ентропія – фундаментальне поняття в теорії інформації, введене основоположником цієї теорії, Клодом Шенноном.

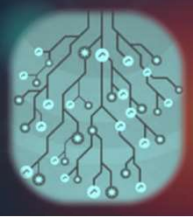
Інтуїтивно під ентропією можна розуміти "ступінь невизначеності" або "ступінь хаосу" у системі. Саме чим впорядкованішою і певнішою є система, тим менша її ентропія. Завдання дерева рішень – внести лад у набір значень цільової змінної. Отже, їх мета – максимальне зменшення ентропії.

Математично ентропія системи з **N** можливими станами обчислюється за формулою Шеннона

$$S = -\sum_{i=1}^{N} p_i \, log_2 p_i$$

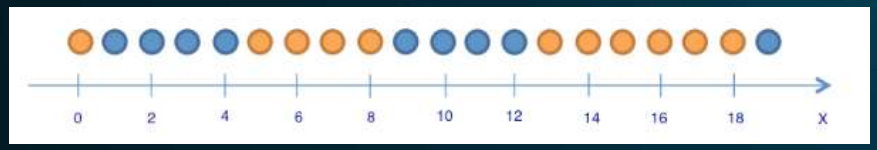
де p_i – ймовірність і-го стану системи.

У певному сенсі поняття ентропії обернене до поняття інформації: чим менше ентропія (невизначеність), тим більше в нас інформації про систему. Введемо поняття "приросту інформації" (information gain) як зменшення ентропії: $IG=S_0-S_1$



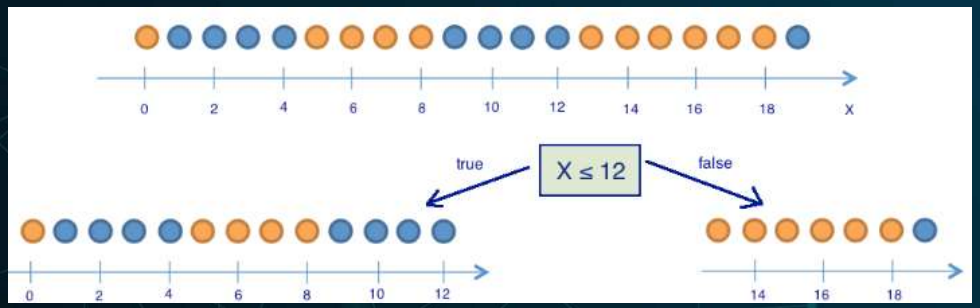
Приклад

Для ілюстрації, як ентропія може допомагає визначити хороші характеристики для побудови дерева рішень, розглянемо такий приклад. Потрібно передбачити колір м'яча, виходячи з його положення.



Є 9 синіх і 11 жовтих куль. Якщо ми навмання витягнемо кульку, то вона з ймовірністю $p_1 = \frac{9}{20}$ буде синього кольору і жовта з ймовірністю $p_2 = \frac{11}{20}$, що дає ентропію $S_0 = -\frac{9}{20}\log_2\frac{9}{20} - \frac{11}{20}\log_2\frac{11}{20} \approx 1$

Перевіримо, як воно зміниться, якщо розділити кульки на дві групи: з позицією, меншою або рівною 12, і більшою за 12.



У лівій групі 13 кульок, 8 синіх і 5 жовтих. Ентропія цієї групи становить $S_1 = -\frac{8}{13}\log_2\frac{8}{13} - \frac{5}{13}\log_2\frac{5}{13} \approx 0.9612$

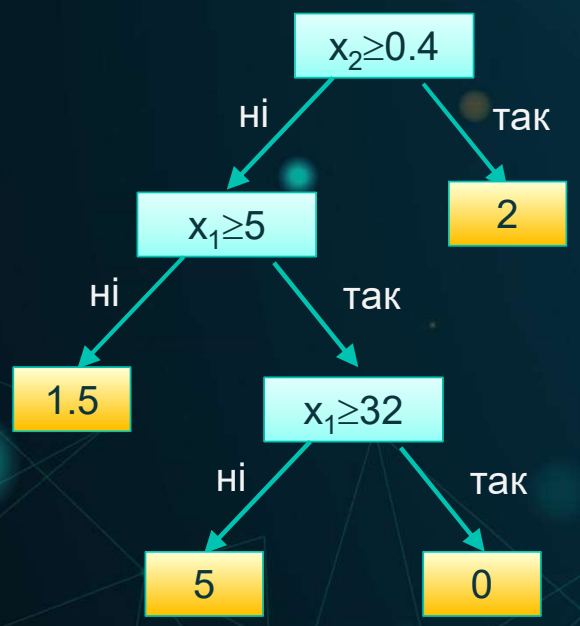
У правій групі 7 куль, 1 синя і 6 жовтих. Ентропія правої групи становить $S_2 = -\frac{1}{7}\log_2\frac{1}{7} - \frac{6}{7}\log_2\frac{6}{7} \approx 0.5917$

Ентропія зменшилася в обох групах, більше в правій групі. Оскільки ентропія відображає ступень хаосу (або невизначеності) у системі, зменшення ентропії називається інформаційним приростом.

Формально, приріст інформації (IG) для розбиття на основі змінної (у цьому прикладі це змінна "$x \leq 12$") визначається як $IG(Q) = S_0 - \sum_{i=1}^{q}\frac{N_i}{N}S_i$, де q – кількість груп після поділу, N_i – кількість об'єктів із вибірки, у якій змінна Q дорівнює i -му значенню.

У розглянутому прикладі розбиття дало дві групи ($q = 2$), одна з 13 елементами ($N_1 = 13$), інша з 7 ($N_2 = 7$). Тому можна обчислити приріст інформації як $IG(x \leq 12) = S_0 - \frac{13}{20}S_1 - \frac{7}{20}S_2 \approx 0.161$

Регресійне дерево

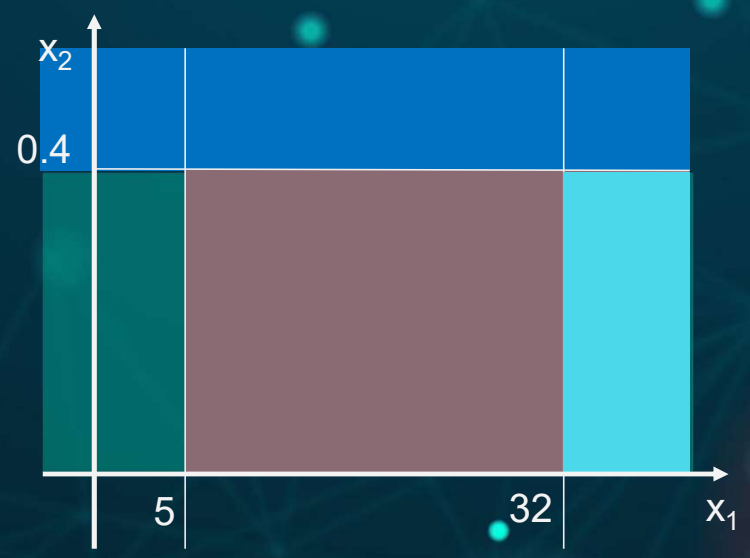


Ознаки:

- x_1: Температура, °C
- x_2: Опади, см/год

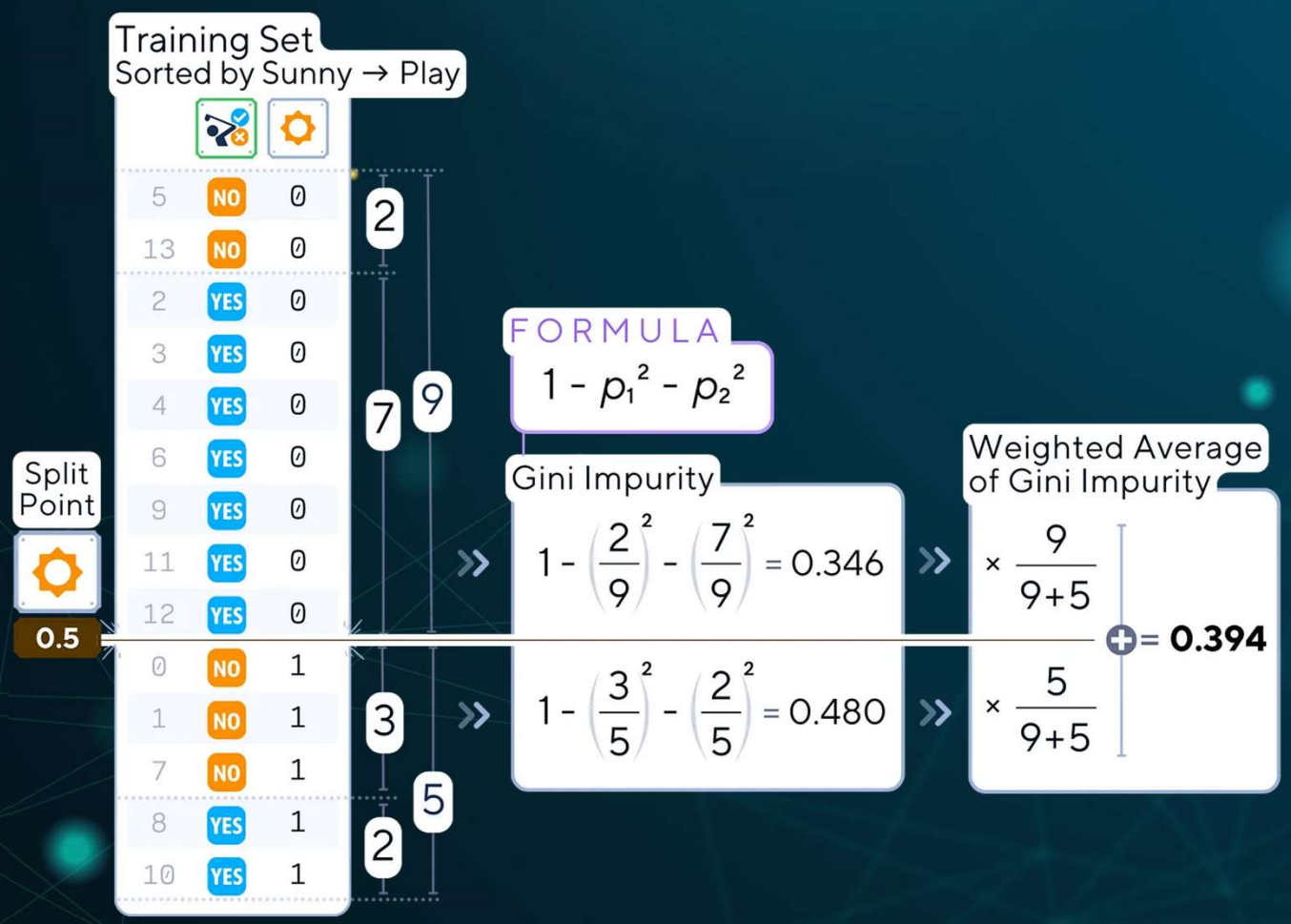
Мітка:

- y: дистанція бігу, км



Дерево розбиває простір ознак по осях

Gini


Training Set
Sorted by Sunny → Play
5 NO 0
13 NO 0
2 YES 0
3 YES 0
4 YES 0
6 YES 0
9 YES 0
11 YES 0
12 YES 0
0 NO 1
1 NO 1
7 NO 1
8 YES 1
10 YES 1
2
7
9
3
2
5
Split Point
0.5
FORMULA
$1 - p_1^2 - p_2^2$
Gini Impurity
$1 - \left(\frac{2}{9}\right)^2 - \left(\frac{7}{9}\right)^2 = 0.346$
$1 - \left(\frac{3}{5}\right)^2 - \left(\frac{2}{5}\right)^2 = 0.480$
Weighted Average of Gini Impurity
$\times \frac{9}{9+5}$
$\times \frac{5}{9+5}$
= 0.394

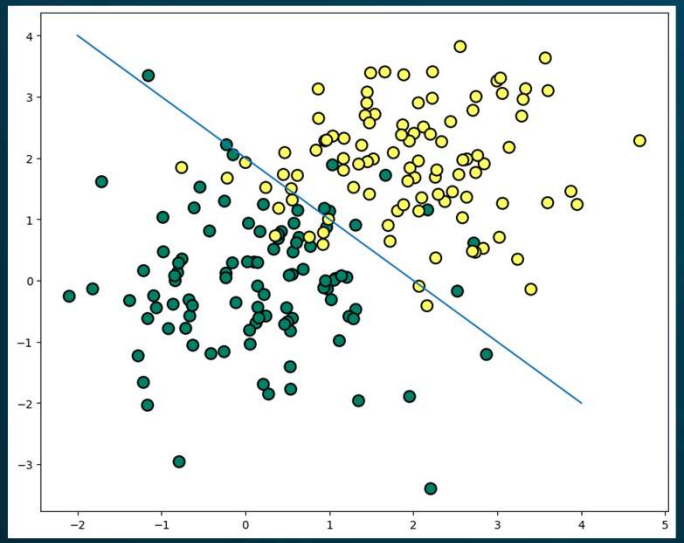
Прикдад реалізації

```python
import numpy as np

# first class
np.random.seed(17)
train_data = np.random.normal(size=(100, 2))
train_labels = np.zeros(100)

# adding second class
train_data = np.r_[train_data, np.random.normal(size=(100, 2), loc=2)]
train_labels = np.r_[train_labels, np.ones(100)]
```

✓ 0.3s

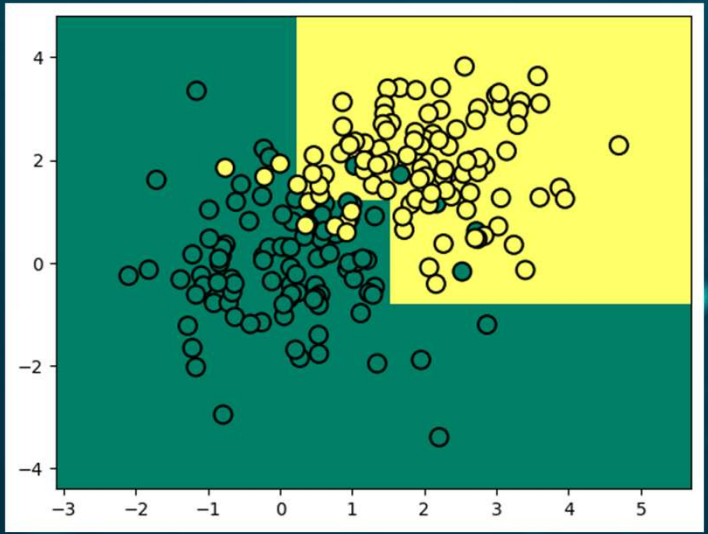
```python
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 8))
plt.scatter(
    train_data[:, 0],
    train_data[:, 1],
    c=train_labels,
    s=100,
    cmap="summer",
    edgecolors="black",
    linewidth=1.5,
)
plt.plot(range(-2, 5), range(4, -3, -1));
```

Розглянемо побудову дерева рішень до деяких синтетичних даних. Згенеруємо зразки двох класів, обидва з нормальним розподілом.

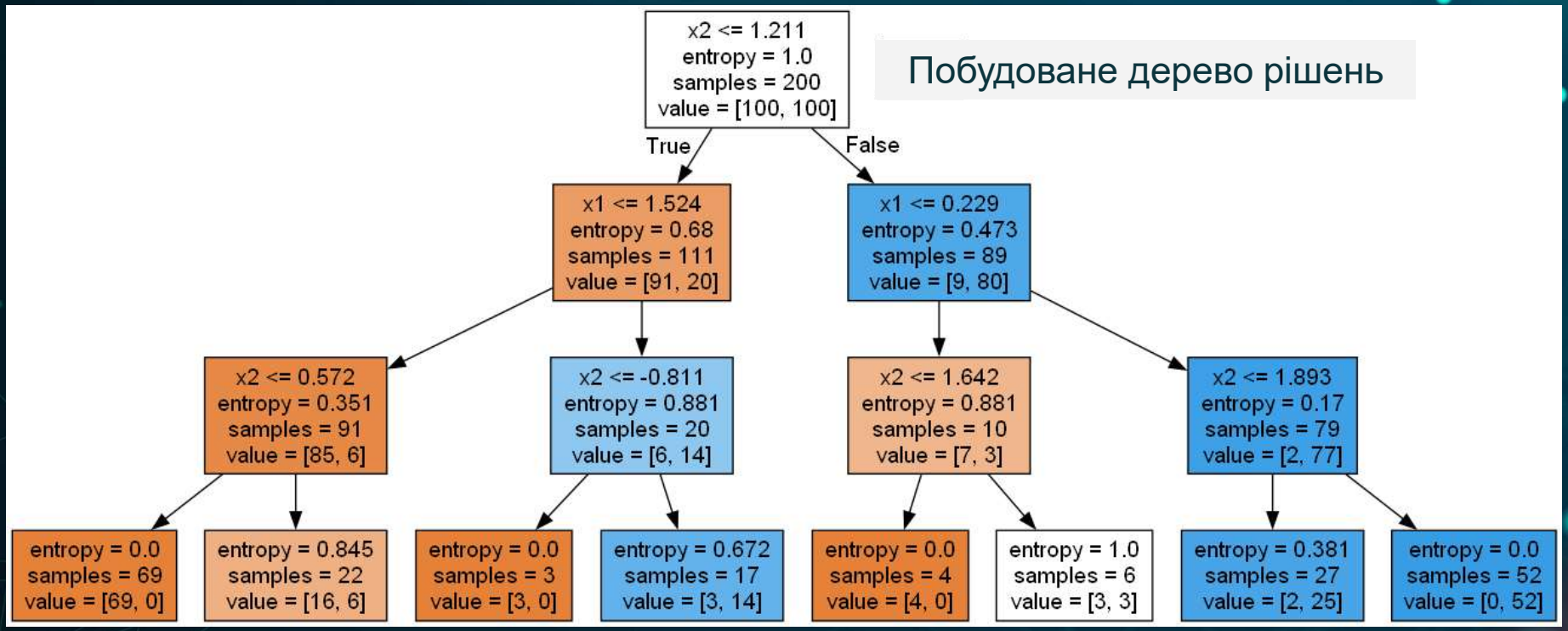


Спробуємо розділити ці два класи за допомогою навчання дерева рішень Sklearn. Будемо використовувати параметр max_depth, який обмежує глибину дерева. Візуалізуємо отриману роздільну межу.

```python
from sklearn.tree import DecisionTreeClassifier


# Let's write an auxiliary function that will return grid for further visualization.
def get_grid(data):
    x_min, x_max = data[:, 0].min() - 1, data[:, 0].max() + 1
    y_min, y_max = data[:, 1].min() - 1, data[:, 1].max() + 1
    return np.meshgrid(np.arange(x_min, x_max, 0.01), np.arange(y_min, y_max, 0.01))


clf_tree = DecisionTreeClassifier(criterion="entropy", max_depth=3, random_state=17)

# training the tree
clf_tree.fit(train_data, train_labels)

# some code to depict separating surface
xx, yy = get_grid(train_data)
predicted = clf_tree.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
plt.pcolormesh(xx, yy, predicted, cmap="summer")
plt.scatter(
    train_data[:, 0],
    train_data[:, 1],
    c=train_labels,
    s=100,
    cmap="summer",
    edgecolors="black",
    linewidth=1.5,
);
```



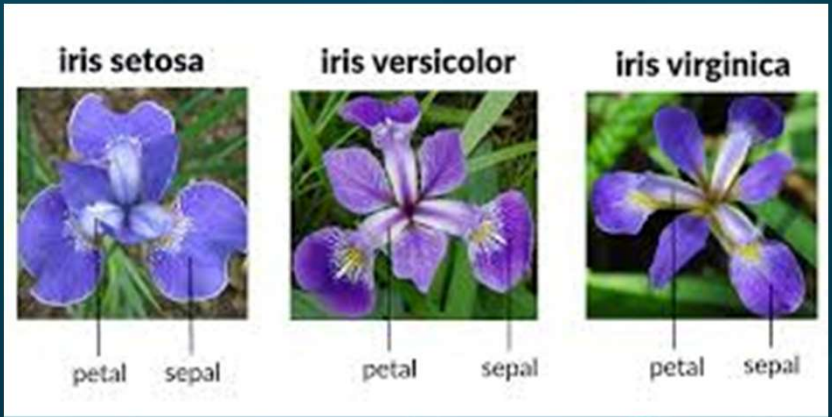
Візуалізуємо саме дерево



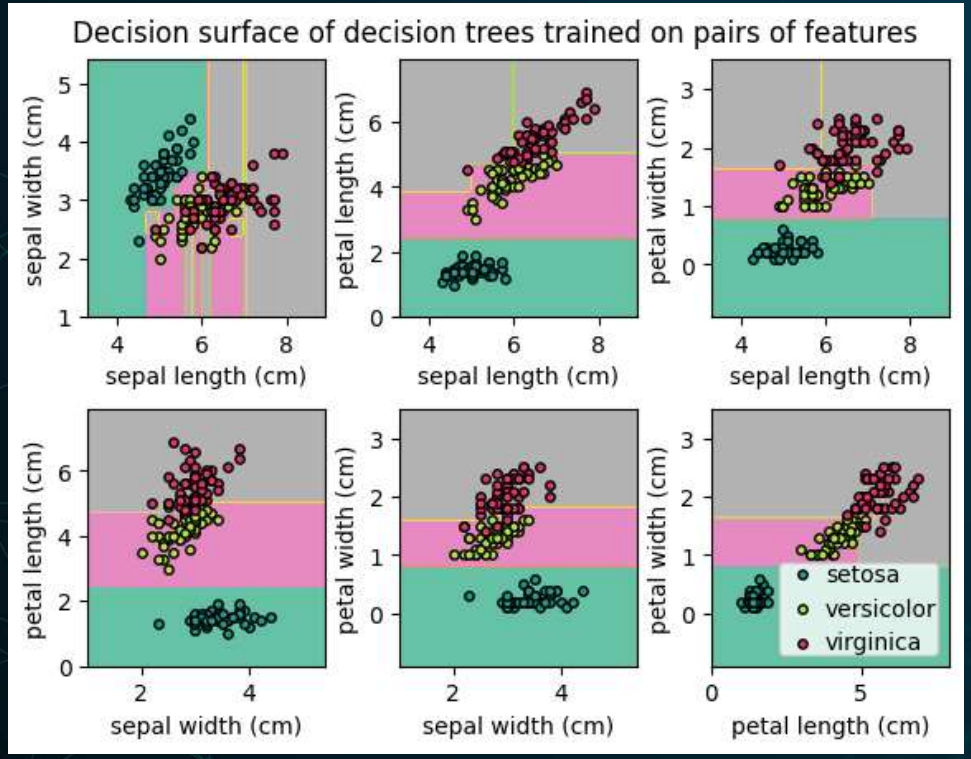
Приклад: Iris dataset

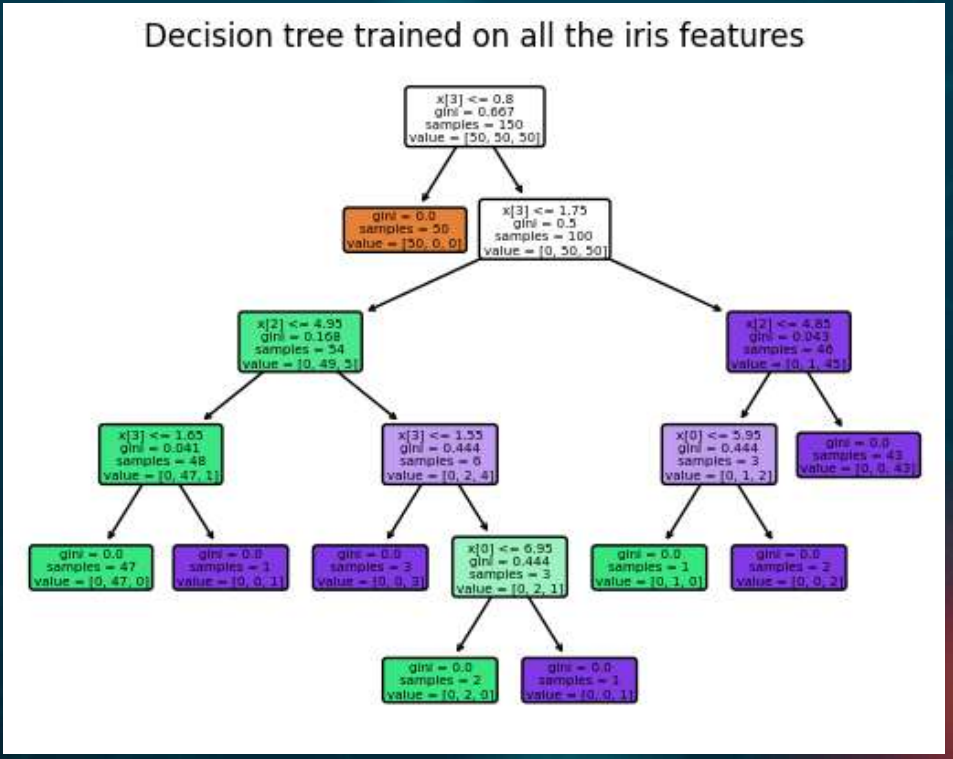
```
from sklearn.datasets import load_iris
iris = load_iris()
```

```
import matplotlib.pyplot as plt
import numpy as np

from sklearn.datasets import load_iris
from sklearn.inspection import DecisionBoundaryDisplay
from sklearn.tree import DecisionTreeClassifier

# Parameters
n_classes = 3

for pairidx, pair in enumerate([[0, 1], [0, 2], [0, 3], [1, 2],
[1, 3], [2, 3]]):
    # We only take the two corresponding features
    X = iris.data[:, pair]
    y = iris.target

    # Train
    clf = DecisionTreeClassifier().fit(X, y)
```



Дерево для ірисів





Random Forest

Random Forest (Випадковий ліс) — це контрольований алгоритм навчання. Він має два варіанти: один використовується для задач класифікації, а інший – для задач регресії.

Це один із найбільш гнучких і простих у використанні алгоритмів. Він створює дерева рішень на основі заданих зразків даних, отримує прогноз з кожного дерева та вибирає найкраще рішення шляхом голосування. Це також досить хороший показник важливості ознак.

Алгоритм Random Forest поєднує кілька дерев прийняття рішень, у результаті чого створюється ліс дерев, звідки й назва Випадковий ліс. У класифікаторі Random Forest більша кількість дерев у лісі призводить до вищої точності.

Алгоритм побудови випадкового лісу складається з наступних етапів:

- ✓ Створити на основі оригінального набору даних декілька штучних наборів.
- ✓ Побудувати окремі дерева
  - ➢ Для побудови нової вершини випадковим чином вибираються «k» ознак із загальної кількості **n** ознак, де **k < n**.
  - ➢ При виборі ознаки для розгалуження вибирають найкращій варіант серед k вершин.

Bagging Метод

Один із способів створити кілька різних моделей — навчити кожну модель за допомогою іншого навчального набору. Метод Bagging (Bootstrap Aggregating) випадковим чином відбирає фіксовану кількість зразків із навчального набору із заміною. Це означає, що точку даних можна намалювати кілька разів.

Data Set 1

x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5

Data Set 2

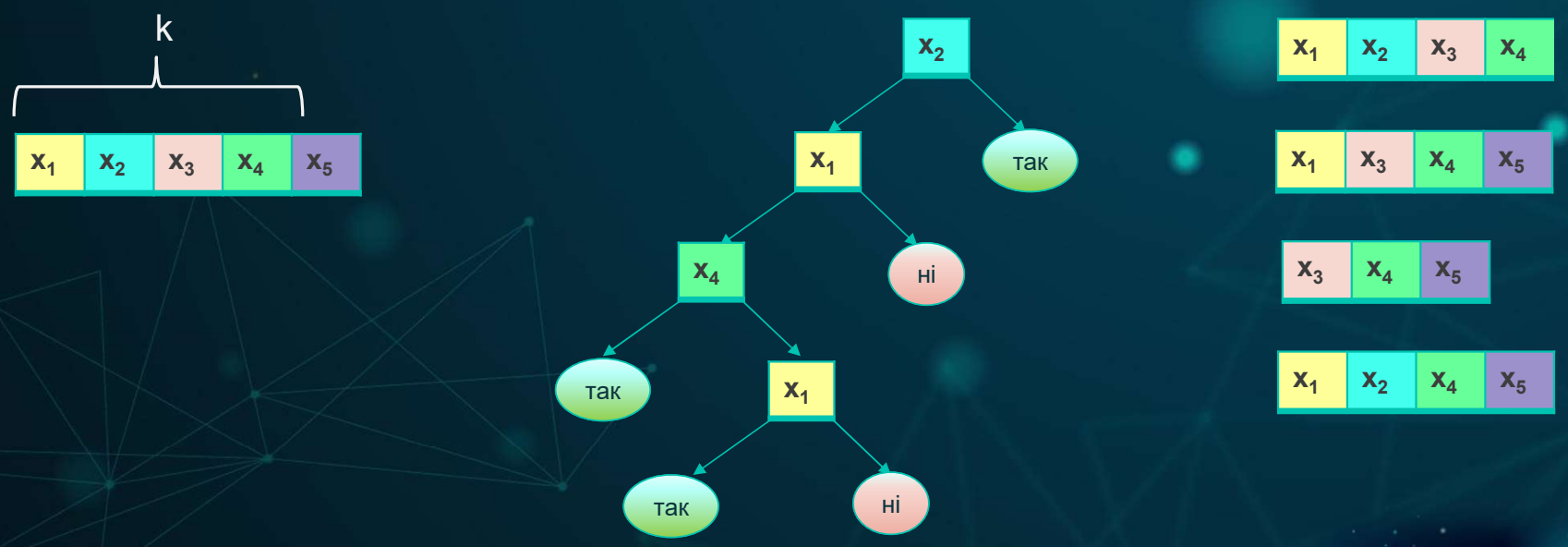
x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5

Data Set 3

x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5
x_1	x_2	x_3	x_4	x_5

Вибір ознак (Feature Selection)

Для кожного розгалуження структури дерева обчислюють найкраще розбиття, використовуючи лише випадково вибрану підмножину ознак. Це ще один спосіб гарантувати, що дерева рішень є якомога різними.



Приклад

```python
from sklearn.ensemble import RandomForestClassifier
clf = RandomForestClassifier(max_depth=4, random_state=0)
clf.fit(X_train, y_train)
```

✓ 0.0s

```
RandomForestClassifier
RandomForestClassifier(max_depth=4, random_state=0)
```

```python
# Make predictions
y2_pred = clf.predict(X_test)
print(y2_pred)
# Evaluate the classifier
print(f"Accuracy: {accuracy_score(y_test, y2_pred)}")
```

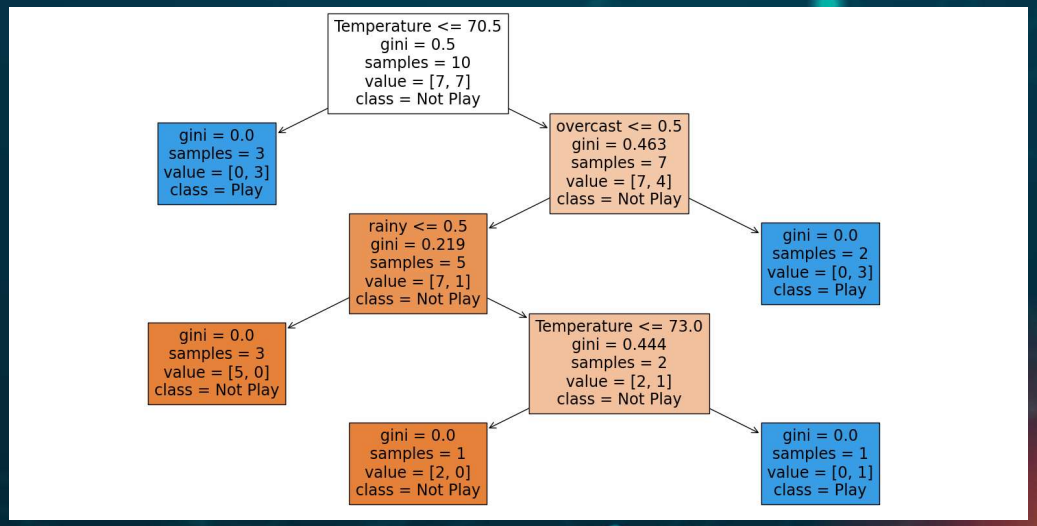
✓ 0.0s

```
[0 1 1 1 0 0 1 1 1 1 1 1 1 1]
Accuracy: 0.8571428571428571
```

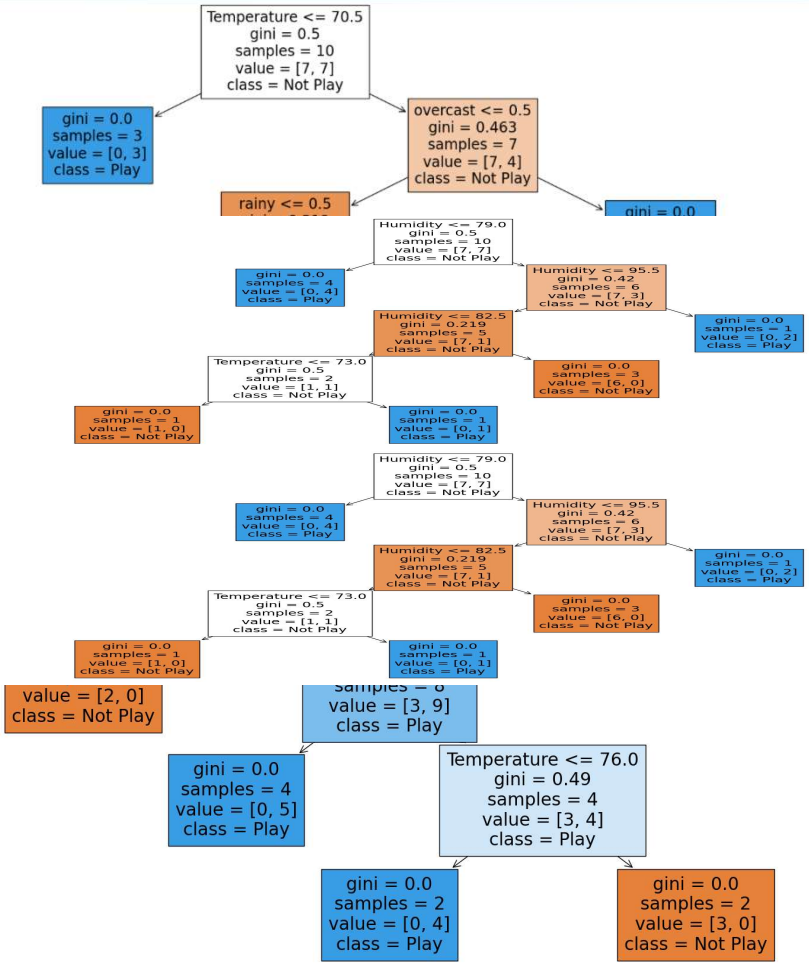
Візуалізація одного з дерев

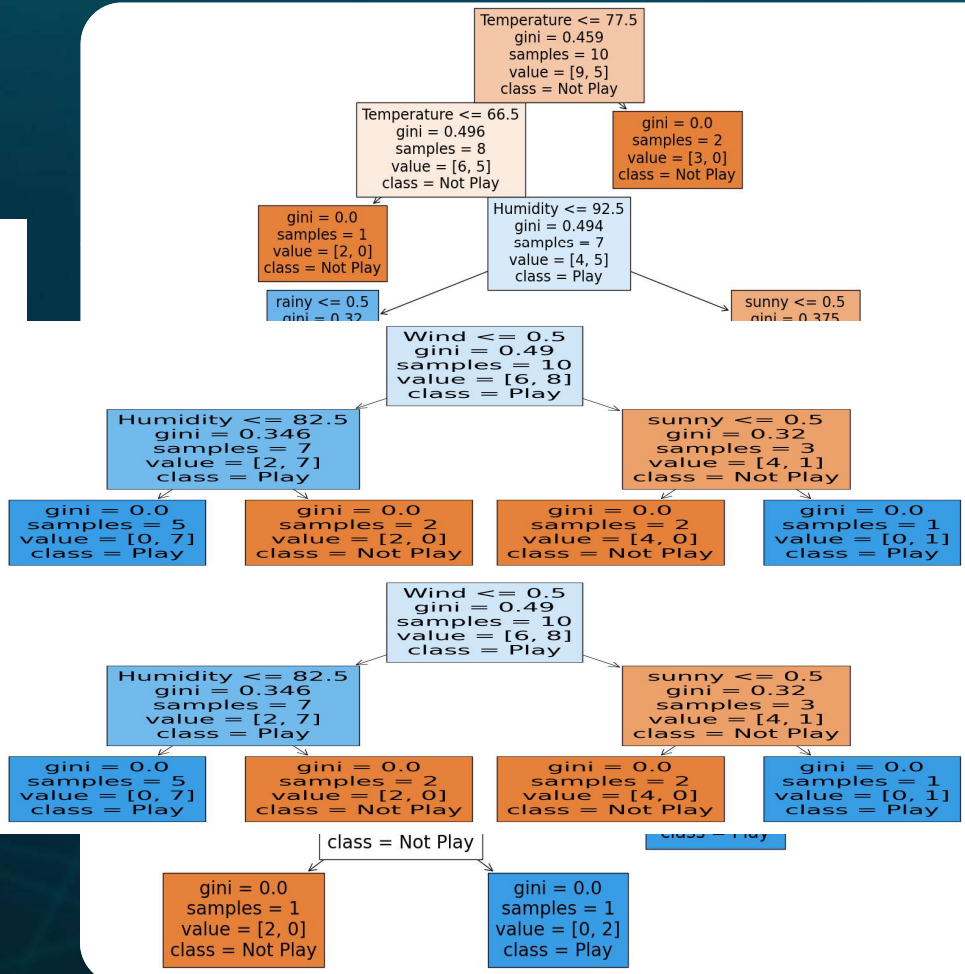
```python
# Plot the decision tree
plt.figure(figsize=(20, 10))
plot_tree(clf.estimators_[1], filled=True, feature_names=X.columns, class_names=['Not Play', 'Play'])
plt.show()
```

✓ 0.1s



Побудовані дерева





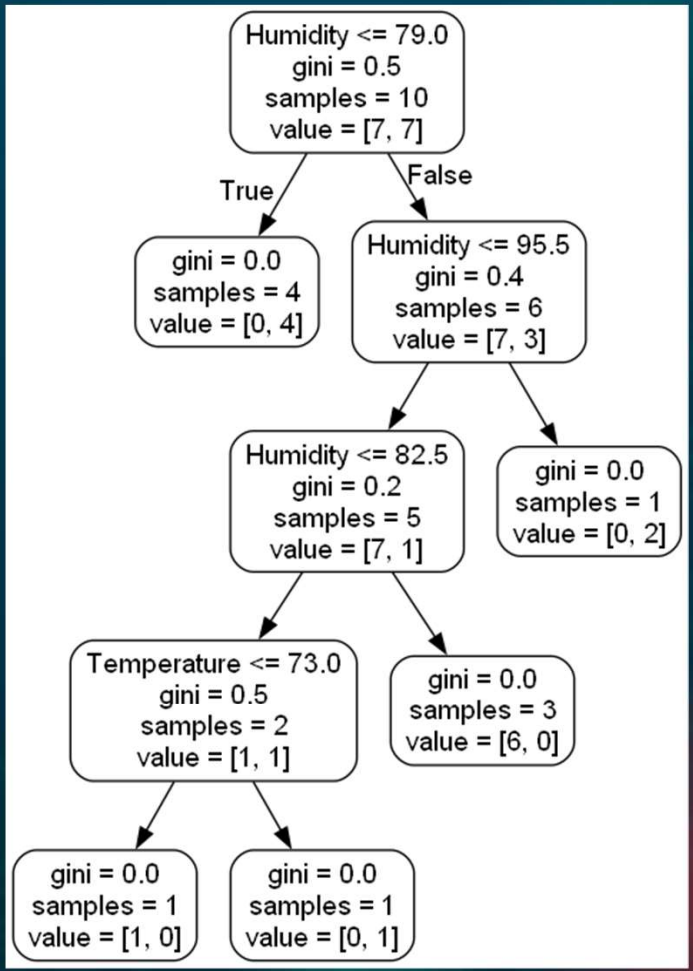
Важливість ознак

```
print(X.columns)
clf.feature_importances_
```

✓ 0.0s

```
Index(['sunny', 'overcast', 'rainy', 'Temperature', 'Humidity', 'Wind'], dtype='object')

array([0.09998781, 0.17022824, 0.07722412, 0.30841558, 0.24223078,
       0.10191346])
```

Інший варіант візуалізації

```
from sklearn.tree import export_graphviz
import pydot


# Pull out one tree from the forest
tree = clf.estimators_[5]# Export the image to a dot file
export_graphviz(tree, out_file = 'tree_rf.dot', feature_names = X.columns, rounded = True, precision = 1)

# Use dot file to create a graph
(graph, ) = pydot.graph_from_dot_file('tree_rf.dot')

# Write graph to a png file
graph.write_png('tree_rf.png')
```

✓ 0.0s